

Tensor Omics: Core Preprocessing, Analysis Pipeline, and Geometric Tools for Semantic Interpretation

Asis Hallab, “el Bobito”

December 1, 2025

Contents

1	Overview	4
2	From Raw Data to Expression Vectors	6
3	Normalization of Expression Values	7
3.1	Variance (Standard Deviation) Stabilization for Scaling	7
3.1.1	LOESS Estimation of Gene-wise Standard Deviation	8
4	Construction of Expression Vector Space	8
4.1	Vector Space Storage in Fortran	8
5	Gene Family Grouping and Centroid Estimation	9
6	Space Interpolation and Smoothing	9
6.1	Grid Step Size Estimation	9
6.2	Gaussian Kernel Interpolation	9
6.3	Gaussian Kernel Smoothing	10
7	Empirical Distribution Estimation	10
7.1	Relative Distance Index (RDI)	10
7.2	Relative Angle Index (RAI)	11
7.3	Relative Axis Signal (RAS)	11
7.4	Rank-normalized Signal Index (RSI)	11
7.5	Relative Frobenius Index (RFI)	12
8	Outlier Detection and Shift Vector Calculation	12
9	Tissue Versatility Calculation	13
10	Clock Plot Projection and Rotation Angle Computation	13
10.1	Efficient Vector Lookup Using K-D Trees	13
11	Geometric Tools for Semantic Interpretation	14
11.1	Direction-Based Subfield Collector	14
11.1.1	When to Normalize (Direction-Only Interpretation)	14
11.1.2	When Not to Normalize (Magnitude Matters)	14
11.1.3	Selection of Directional Subfield	15
11.2	Signal-Based Subfield Detection	15
11.2.1	Vector Collection	15
11.2.2	Magnitude Distribution and Noise Filtering	15
11.2.3	Region Growing	16

11.2.4	Region Marking and Continuation	16
11.2.5	Interpretation	16
11.3	Workbench for Interactive Exploration	16
12	Time Series Analysis	16
12.1	Stage 1: Per-Gene Trajectory Interpolation	17
12.2	Stage 2: Contextual Smoothing Across Gene Families	17
12.3	Adaptive Kernel Width	17
12.4	Shift Vector Construction	17
12.5	Angular Projection and Clock Plot Modes	18
12.6	Clock Plot Visualization Modes	18
12.7	Angular Shift Velocity (ASV)	18
12.8	Mjöltnir’s Handle — Canonical Axis Reconstruction for Eitri Surface Extraction	18
12.9	Instantaneous Acceleration (Future Extension)	19
12.10	Biological Interpretation	19
12.11	Implementation Notes	19
13	Gilgamesh Learning: Semantic Field-Aware Pattern Classification	20
13.1	Overview	20
13.2	Motivation and Biological Context	20
13.3	Method	20
13.3.1	Cylindrical Binning Along Mjöltnir’s Handle	20
13.3.2	Canonical Descriptors	20
13.3.3	Training and Classification	21
13.4	Explainability	21
13.5	Semantic Alignment and Orientation of Subfields	21
13.6	Glyph Description via Lexical Metadata	21
13.7	Biological Applications	22
13.8	Advantages Over Conventional ML	22
13.9	Example Pseudocode (Sketch)	22
14	Shape truthful clustering	22
14.1	Summary	22
14.2	Interpretation by Vector Type	23
14.2.0.1	Expression vector fields (use spheres):	23
14.2.0.2	Shift vector fields (use cones):	23
14.3	Verdict	23
14.4	Note	23
15	Detection of Subfunctionalization and Dosage Effects via Dynamic Programming	23
15.1	Overview	23
15.2	Data Structures	24
15.3	Algorithm Steps	24
15.3.0.1	Initialization	24
15.3.0.2	Dynamic Programming Iteration	24
15.3.0.3	Final Filtering	25
15.4	Notes on Implementation	25
15.5	Pseudocode	25
16	Trajectory-Based Signed Contribution Analysis	25
16.1	World Bank Example Dataset	25
16.2	Trajectory related scientific questions	26
16.3	Covariance-Based Contribution Decomposition	26
16.4	Linear Combinations of Factors and Dependents	26
16.5	Baseline Modes	27

16.5.1	Raw Contributions	27
16.5.2	Minimum-Centred Contributions	27
16.5.3	Mean-Centred Contributions	27
16.6	Optional Inter-Trajectory Normalisation	27
16.7	Permutation Tests	27
16.8	Comparisons Between Groups of Trajectories	28
16.9	Visualisation Strategies	28
16.10	Discussion	28
16.11	Velocity- and Acceleration-Based Contribution Analysis	28
16.11.1	Velocity Contributions	28
16.11.2	Acceleration Contributions	29
16.12	RAP-Based Trajectory Contributions	29
16.13	Group-Level Trajectory Field Descriptors	29
17	Module Descriptors (Tensor-Field Descriptors)	31
17.1	Concept of Modules (Sets of Vectors) and Vector Fields	31
17.2	Four Descriptors of a Module / Set of Vectors	31
17.2.1	Energy (Signal Strength)	31
17.2.2	Rank (Dimensionality or Spread)	32
17.2.3	Principal Flow Direction	32
17.2.3.1	Geometric Principal Axis (from SVD of M).	32
17.2.3.2	Dynamical Principal Flow (from EigenDecomposition of C).	32
17.2.3.3	Interpretative Complementarity.	32
17.2.3.4	Mathematical Note on Coincidence of Eigenvectors (singular vectors)	33
17.2.4	Rotation Axis and Magnitude	33
17.2.4.1	Approximate Rotation via the Antisymmetric Component of M	33
17.3	Conclusion of the four module descriptors	34
17.4	Normalization of module descriptors	34
17.5	Vector Sets versus Vector Fields	34
17.6	Tracing a Module along its Semantic Axis	35
17.7	Interpretation and Scientific Questions	35
18	Field Surface and Volume Inference (FiSure, “Eitri”)	35
18.1	Motivation and Concept	35
18.2	Algorithmic Outline	35
18.3	Interpretation of FiSure Quantities	36
18.4	Applications and Interpretation	37
18.5	Summary	37

1 Overview

The Tensor Omics (TOX) tool implements Geometric Learning in a semantic vector space. We represent the data in its natural space, where each measurement is its own axis. We use model-free and minimum-parameter geometric methods to describe the data, find meaningful patterns, identify significant contributions of factors to dependent variables, analyse trajectories, and learn fields of vectors with intrinsic similarity.

We rely exclusively on pure linear algebra to characterize these structures, using distances, angles, projections, and linear mapping.

Tensor Omics differs from conventional geometric learning tensor decomposition in that it operates in empirically defined semantic vector spaces rather than abstract latent dimensions — a demonstration (See Figure 1):

(A) Medical omics vectors in semantic space. Each gene, protein or clinical patient measure is placed as a point in multi-dimensional space, with axes representing gene expression in a tissue or clinical patient measures. Arrows denote shift vectors from gene family centroids and enable the explainable detection of changes in gene expression from healthy control. Grid points support interpolation and smoothing. Patient time trajectories are shown in the lower left (healthy controls in green, mean control trajectory in fat light green, and a cancer trajectory in orange). The disease specific signals can directly be obtained by geometric trajectory comparison (violet time resolved shift arrows $\vec{s}_{t_i}$). Significant signals are detected empirically as outliers in the upper 5%-ile.

(B) Angle to the space diagonal as a measure of omics specificity: low angles indicate e.g. broad tissue expression, high angles signal strong tissue preference.

(C) Clock hand angle in the Relative Axes Plane (RAP), capturing changes in tissue preference with robust correction for the overall expression magnitude.

(D) Causal and explainable omics learning by selecting factors showing geometric similarities in predictive properties. Tensor Omics supports three independent strategies: radial subset growing until only background signal remains (orange), metadata-based selection (e.g. inflammatory response, green), and directional filtering by alignment with a user-defined direction of interest (DOI, e.g. median survival time after treatment or distance of patient trajectory to the healthy control, blue). Methods can be combined to define coherent subfields for geometric analysis.

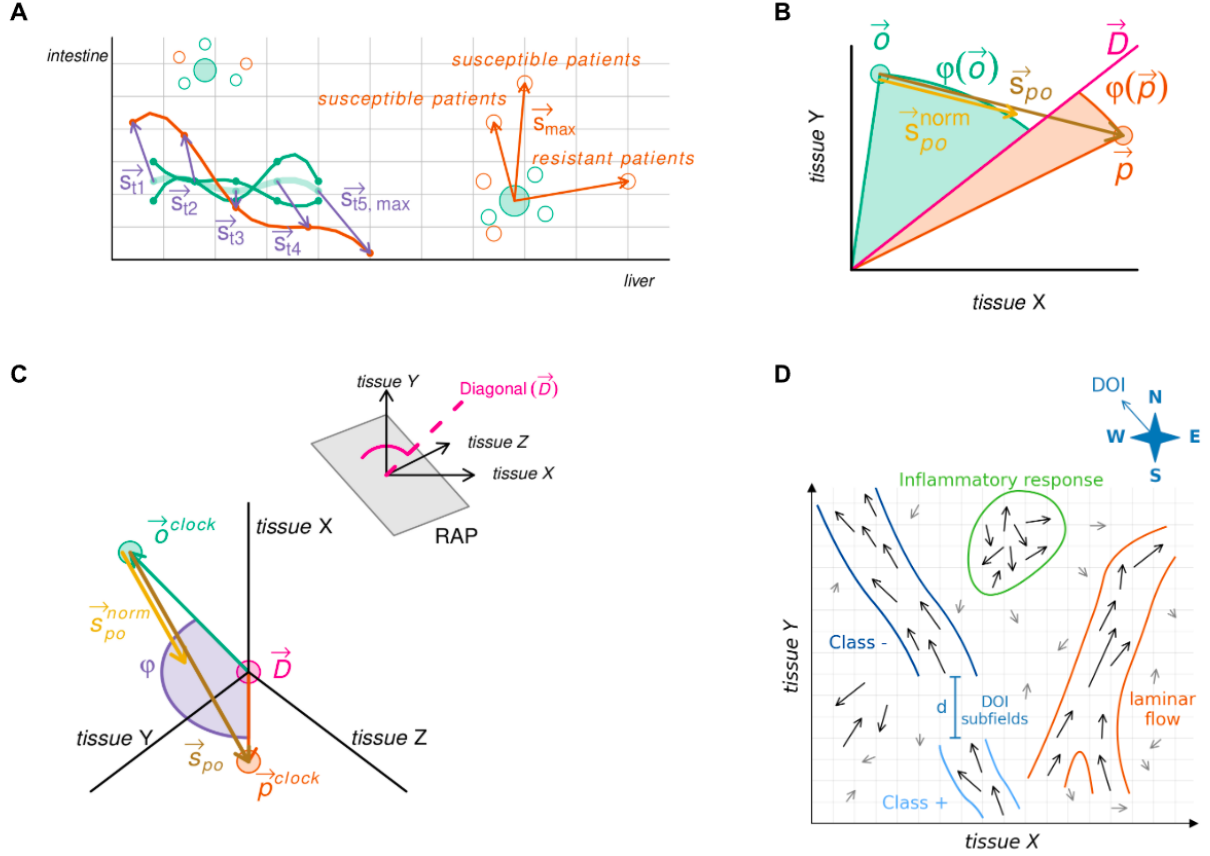
(E) Explainable analysis of patient time trajectories (as shown in A). Changes in tissue preference (angles ϕ) relative to the anchor axis (AA). Inset plots show the distance to the healthy control trajectory (top) and the angle between consecutive shift vectors (bottom).

(F) Learned subfield (see D) characterization by emulating the aggregated effect of vectors on a virtual particle. Measures include signal strength (Frobenius norm $\|M\|_F$), disease subtyping as directional spread (rank), potential disease specific biomarkers as principal flow direction (first eigenvector of covariance matrix C), and disease specific changes in omics factors as rotation axis and magnitude (from the antisymmetric component $0.5 \times (C - C^T)$).

(G) Gradient-following kernel traversal to elucidate disease trajectory analysis in early, mid, and late stages: subfield (see D) structure is decomposed via Singular-Value-Decomposition (SVD) into dominant axes (E1, E2 top left) which are smoothed into a single semantic axis (pink line), along which kernel profiles (pink bell top right) reveal flow, rotation, spread, and signal strength (as in F; lower left). Embedding of subfields for machine learning (lower right): The semantic axis is used to orient the subfield with the major flow of the vectors ($\oplus$) and split into five-percentile windows pushed along the semantic axis (dotted lines). Each section is represented by an aggregated vector of field strength (norm), directional spread (rank), principal direction (eigenvector), and rotational moment.

(H) The semantic axis (see G) is used to push a higher dimensional object through a lower dimensional space for visualization in the “Data-Vis” interactive graphical user interface (Tensor Omics package). This enables direct visualization of higher dimensions and can be used to show for example the developmental hourglass.

Panels A–H together illustrate how Tensor Omics unifies geometric learning, empirical explainability, and visualization within a single analytical framework.



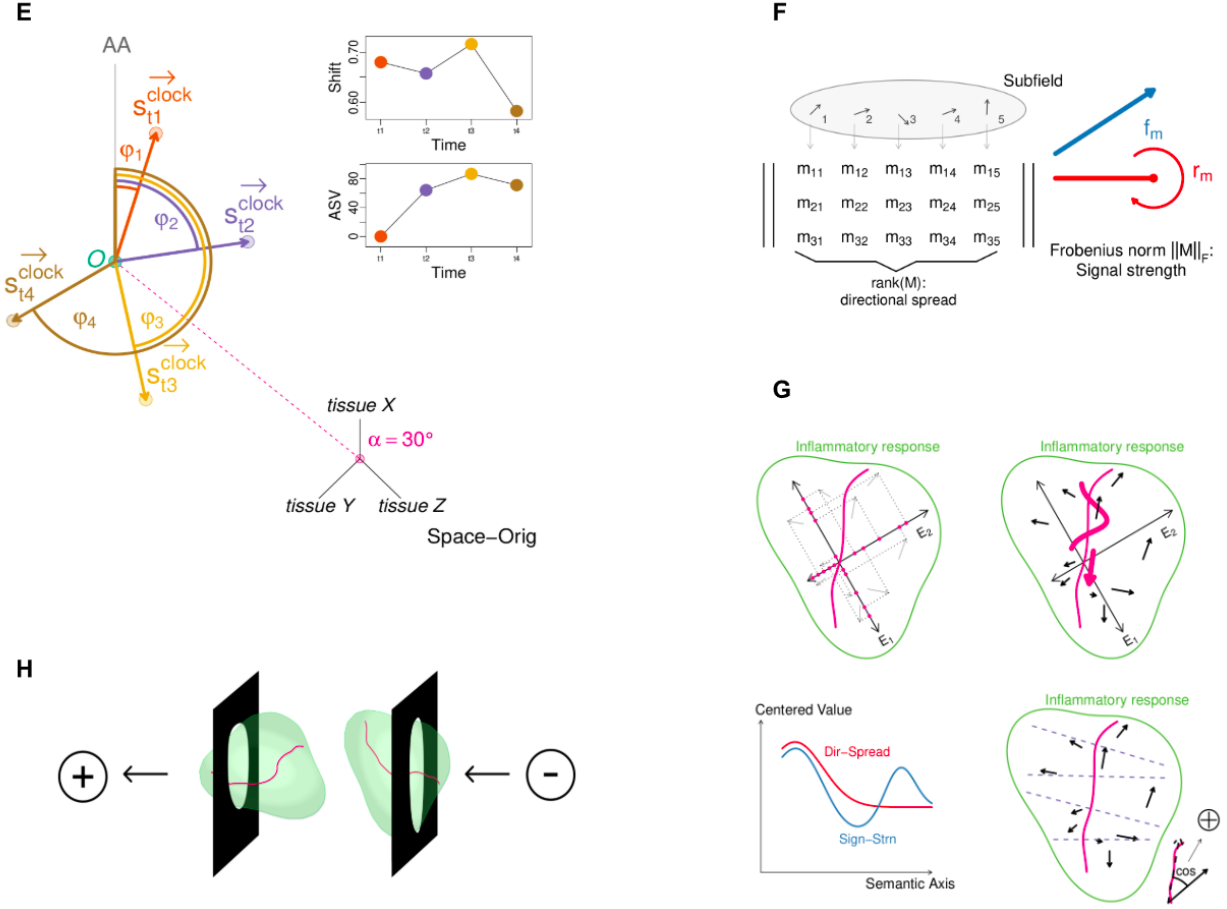


Figure 1: Overview of the Tensor Omics

2 From Raw Data to Expression Vectors

The starting point for all downstream analyses is a table of gene-wise expression measurements across a set of biological samples, such as tissues, developmental stages, or experimental conditions. These measurements may originate from transcriptomic, proteomic, or metabolomic experiments, typically normalized to account for sequencing depth and technical variation.

What is Gene Expression Data? For each gene, a real-valued quantity is recorded that reflects its biological activity in a given sample. In the case of transcriptomics, this is often the abundance of messenger RNA (mRNA) molecules transcribed from that gene, estimated via sequencing and mapped back to the gene’s known sequence. The result is a non-negative real number proportional to the gene’s transcriptional activity in that sample. After normalization and transformation, these values can be treated as coordinates in a real vector space.

Thus, each gene is represented as a vector $\vec{v} \in \mathbb{R}^n$, where n is the number of biological samples (e.g., tissues or time points), and each component $v_i \geq 0$ corresponds to the expression level in sample i . These vectors encode the distribution of gene activity across conditions and form the geometric basis for all downstream analyses.

Note on Stress Responses. In certain studies, a subset of the samples may correspond to stress conditions. Here, “stress” refers to any external or internal perturbation—such as mechanical damage, heat shock, pathogen infection, or nutrient deprivation—that triggers measurable changes in gene expression. These are often analyzed relative to baseline conditions using log fold-change transformations, such that the resulting vectors reflect differential expression patterns, including up- and downregulation across dimensions.

3 Normalization of Expression Values

Raw gene expression data is normalized in the following steps:

1. Division by gene-wise standard deviation for scale normalization. For estimation of the correct standard deviation in low dimensional data-sets see below (3.1).
2. Quantile normalization across samples to reduce global expression biases.
3. Log-transformation: $x \mapsto \log(1 + x)$ to stabilize variance and suppress outliers.
4. For stress response studies, log fold-changes are computed:

$$\Delta_{\text{stress}} = \log(1 + x_{\text{stress}}) - \log(1 + x_{\text{control}})$$

This effectively makes each stress related axis show the fold change in gene expression under stress, e.g. “drought in root divided by control root”.

3.1 Variance (Standard Deviation) Stabilization for Scaling

In Tensor Omics, normalization of expression vectors requires division by gene-wise standard deviation (SD) across samples. However, empirical SD estimates are unstable for small sample sizes. To address this, we implement a dimension-dependent stabilization strategy that improves robustness while maintaining geometric interpretability.

The strategy is defined as follows:

- For small sample sizes $n < 10$:
 - Empirical SD estimates are highly unreliable.
 - Therefore, for each gene, the standard deviation is replaced by a LOESS-fitted value (see 3.1.1).
 - The LOESS curve is fitted to model the relationship between gene-wise mean expression and gene-wise empirical SD across all genes.
- For intermediate sample sizes $10 \leq n < 21$:
 - A weighted combination of empirical SD and LOESS-fitted SD is used.
 - The effective standard deviation is computed as

$$\text{SD}_{\text{effective}} = \lambda \times \text{SD}_{\text{empirical}} + (1 - \lambda) \times \text{SD}_{\text{LOESS}},$$

where $\lambda = (n - 10)/11$ increases linearly from 0 at $n = 10$ to 1 at $n = 21$.

- For large sample sizes $n \geq 21$:
 - Empirical SD estimates are considered sufficiently stable.
 - Pure empirical SD is used without adjustment.

This stabilization ensures that scaling by SD is consistent and resistant to noise across a wide range of dataset sizes. It preserves the empirical geometric structure of the data while suppressing variance artifacts that would otherwise distort Tensor Omics analyses, particularly in small or medium sample regimes.

3.1.1 LOESS Estimation of Gene-wise Standard Deviation

To obtain stabilized estimates of gene-wise standard deviations for small sample sizes, Tensor Omics applies a LOESS smoothing procedure based on the relationship between gene-wise mean expression and gene-wise empirical SD.

The procedure consists of the following steps:

- Calculate the empirical mean and empirical standard deviation for each gene across all samples.
- Plot each gene as a point in a scatter plot with
 - X-axis: gene-wise mean expression,
 - Y-axis: gene-wise empirical standard deviation.
- Fit a LOESS curve to these points to model the typical dependence of standard deviation on mean expression.
- For each gene, given its mean expression value, predict its standard deviation by evaluating the fitted LOESS curve at the gene’s mean.

This method exploits the fact that gene-wise mean expression values are substantially more stable than standard deviation estimates, especially at low sample sizes. While the empirical standard deviation suffers from high variability when few data points are available, the mean value remains a more robust and lower-variance statistic. By basing the standard deviation estimation on the more reliable mean, the procedure suppresses noise-induced artifacts and yields smoothed, biologically plausible variance estimates. This LOESS-based adjustment is critical for preserving the integrity of the normalization process in small-sample regimes, ensuring that subsequent geometric analyses remain meaningful and reproducible.

4 Construction of Expression Vector Space

- Biological replicates belonging to the same axis (tissue, stressed tissue, etc.) are averaged post-normalization.
- Each gene is represented as a vector $\vec{p} \in \mathbb{R}^n$, where n is the number of conditions or tissues. Hence the analysis is embedded in an n -dimensional ambient space.
- Expression vectors are stored in Fortran arrays. *Importantly* Fortran is column major and vectors *must* be columns.
- Two K-D Trees are constructed:
 - A raw-space K-D Tree (unnormalized vectors) for Euclidean queries.
 - A normalized-space K-D Tree (unit-length vectors) for cosine similarity and angular queries.
- The first axis is designated the **Anchor Axis (AA)** for visualization.

4.1 Vector Space Storage in Fortran

All expression vectors in Tensor Omics are stored in a central memory structure optimized for performance and access in Fortran. This architecture ensures efficient storage, lookup, and downstream computation.

- **vec_store:** A two-dimensional Fortran array of size $n \times v$, where n is the number of expression dimensions (e.g., tissues or conditions), and v is the number of vectors (e.g., genes). Each column represents one expression vector.
- **cart_tree:** A K-D Tree constructed on the raw (unnormalized) vectors stored in **vec_store**. This structure enables rapid spatial neighborhood queries using Euclidean distance in the original expression space. This is not initialized until step [10.1](#).

- **sphere_tree**: A second K-D Tree constructed from unit-length normalized vectors derived from `vec_store`. This structure supports directional queries based on cosine similarity or angular distance. This is not initialized until step 10.1.

Together, these data structures allow both magnitude-sensitive and direction-sensitive operations across expression fields.

5 Gene Family Grouping and Centroid Estimation

- Gene families, populations, or functional groups are defined using metadata.
- Family centroids $\vec{o}$ are computed either as:
 - The mean vector of all family members.
 - The mean vector of a family’s orthologs only.
- All metadata and vectors can be serialized into `.txdata` binary files.

6 Space Interpolation and Smoothing

To support field-based analysis and ensure consistent representation across expression space, a regular structured grid is constructed. This grid forms the structural foundation for kernel-based interpolation and vector field smoothing.

6.1 Grid Step Size Estimation

The grid resolution is determined empirically from intra-family expression variation. Specifically, for each gene family, the Euclidean distance between each member’s expression vector and the corresponding family centroid is computed. These intra-family distances are aggregated across all families into a single distribution. The grid step size (e_g) is then set as the **first quartile** (25th percentile) of this distribution.

Note: Since this step requires family membership and centroid information, it must be performed after gene family grouping and centroid estimation (see Section 5).

6.2 Gaussian Kernel Interpolation

After grid construction, expression vectors are propagated onto the surrounding grid using a Gaussian kernel function centered at each grid vector’s position. This operation produces a smoothly interpolated vector field that approximates local expression structure in the semantic space.

For a given expression vector $\vec{v}$ and a grid point $\vec{g}$, the interpolation weight is computed as:

$$w(\vec{v}, \vec{g}) = \exp\left(-\frac{\|\vec{v} - \vec{g}\|^2}{\sigma^2}\right), \text{ with } \sigma = e_g, \text{ the grid-step-size}$$

Only expression vectors located within a cutoff radius of $R = 3\sigma$ from the grid point contribute to its interpolated value. This follows standard statistical convention: for Gaussian kernels, 99.7% of the total mass lies within three standard deviations. This choice ensures inclusion of nearly all local signal while excluding remote vectors that would introduce unrelated semantic influences.

The kernel width σ is set equal to the estimated grid step size, aligning spatial influence with the empirically defined resolution of expression variability. Each grid point $\vec{g}$ receives the kernel-weighted average of all expression vectors within its support radius:

$$\vec{v}_{\text{interp}}(\vec{g}) = \frac{\sum_{\vec{v} \in \mathcal{V}_g} w(\vec{v}, \vec{g}) \cdot \vec{v}}{\sum_{\vec{v} \in \mathcal{V}_g} w(\vec{v}, \vec{g})}$$

where $\mathcal{V}_g$ is the set of all original expression vectors within range of $\vec{g}$. No interpolated vectors are used in this step; only original gene expression vectors contribute.

6.3 Gaussian Kernel Smoothing

After interpolation, a secondary smoothing step is applied to reinforce field coherence. This step uses the same Gaussian kernel formulation but is conceptually and operationally distinct from interpolation.

In smoothing, both the interpolated grid vectors $\vec{v}_{\text{interp}}$ and the original expression vectors $\vec{v}_{\text{orig}}$ contribute to the smoothed result. However, smoothing is applied only to the interpolated grid points, and the smoothed vectors are *not* recursively used in further smoothing steps. This design ensures mathematical clarity, avoids progressive drift or oversmoothing, and enables efficient parallel computation.

The smoothed vector at grid point $\vec{g}$ is computed as:

$$\vec{v}_{\text{smooth}}(\vec{g}) = \frac{\sum_{\vec{v} \in \mathcal{V}_g \cup \mathcal{G}_g} w(\vec{v}, \vec{g}) \cdot \vec{v}}{\sum_{\vec{v} \in \mathcal{V}_g \cup \mathcal{G}_g} w(\vec{v}, \vec{g})}$$

where:

- $\mathcal{V}_g$ are original expression vectors within $R = 3\sigma$ of $\vec{g}$,
- $\mathcal{G}_g$ are neighboring interpolated grid vectors (e.g., in a Moore neighborhood),
- All weights are computed using the same Gaussian kernel with width σ .

Importantly, smoothed vectors are not used to further smooth neighboring grid points. This avoids recursive blending and ensures that the smoothing operation can be executed independently across grid points, supporting massive parallelization on CPU or GPU backends.

Clarification regarding mathematical consistency: This two-step method preserves both the semantic geometry and the quantitative scale of the original expression space. Interpolation is based solely on raw expression vectors, and smoothing includes original anchors, thereby preventing topological drift. The choice of $R = 3\sigma$, combined with symmetric Gaussian kernels and fixed kernel width, ensures that no bias is introduced into the vector field and that all operations remain invariant under rotation and translation in expression space.

Interpretation: Interpolation constructs a continuous field over grid coordinates. Smoothing reinforces continuity and prepares the field for robust downstream operations such as flow estimation, rotation analysis, or module surface detection.

7 Empirical Distribution Estimation

To enable robust outlier detection and hypothesis-free exploration, Tensor Omics constructs empirical distributions of core metrics across all gene families.

7.1 Relative Distance Index (RDI)

For each gene family, we define the centroid $\vec{o}$ as the mean of all ortholog vectors $\vec{o}_i$ across species. To assess the magnitude of a gene’s divergence from this conserved expression center, we compute the Euclidean distance:

$$d = \|\vec{p} - \vec{o}\|$$

where $\vec{p}$ is the expression vector of the gene being evaluated.

To normalize this distance and enable scale-independent comparison across families, we compute a family-specific scaling factor as the maximum distance between any ortholog vector and the ortholog centroid:

$$d_{\text{scale}} = \max_i \|\vec{o}_i - \vec{o}\|$$

This normalization is linear in the number of orthologs and avoids the quadratic cost of pairwise distance computation. The Relative Distance Index (RDI) is then defined as:

$$\text{RDI} = \frac{\|\vec{p} - \vec{o}\|}{d_{\text{scale}}}$$

In cases where ortholog annotations are unavailable or the family contains only a single ortholog, a statistical smoothing strategy is used to estimate an appropriate scaling factor. Specifically, for each gene family, we calculate:

$$FS_i = \text{median}_j \left(\|\vec{f}_j - \vec{o}_i\| \right) \quad \text{and} \quad FD_i = \text{sd}_j \left(\|\vec{f}_j - \vec{o}_i\| \right)$$

where $\vec{f}_j$ are the expression vectors of all family members and $\vec{o}_i$ is the family centroid (mean of all $\vec{f}_j$). A smoothing function $f(\cdot)$, e.g. via LOESS, is fit to the set $\{(FS_i, FD_i)\}$ across all families. This yields an empirical mapping from median distance to expected spread. For families lacking orthologs, the smoothed value $f(FS)$ is used as the denominator in:

$$RDI_{\text{smoothed}} = \frac{\|\vec{p} - \vec{o}\|}{f(FS)}$$

This hybrid strategy ensures that RDI remains well-defined, interpretable, and comparable across evolutionary, functional, and statistical use cases in Tensor Omics.

All RDI values are pooled across all families to build an empirical distribution of relative intra-family divergence. This enables percentile-based filtering of significant shifts (e.g., 95th percentile for outlier detection) without requiring parametric assumptions.

Note: While RDI values are used to identify significant divergence, the unnormalized distances d_i are retained for all downstream geometric computations, such as vector field construction and shift vector analysis.

7.2 Relative Angle Index (RAI)

The angular difference between a vector $\vec{p}$ and its centroid $\vec{o}$ is computed using the cosine formula:

$$RAI = \cos^{-1} \left(\frac{\vec{p} \cdot \vec{o}}{\|\vec{p}\| \cdot \|\vec{o}\|} \right)$$

This identifies divergence in direction. Thresholds (e.g. 95%) are derived from the pooled distribution of angles.

7.3 Relative Axis Signal (RAS)

For each unit-normalized expression vector $\vec{p}^{\text{norm}}$, we consider its component $\vec{p}_i^{\text{norm}}$ along each axis i (e.g., tissue). For each axis, the 5th percentile across all vectors is computed as the inactivity threshold. A gene is considered neofunctionalized in axis i if:

$$\vec{o}_i^{\text{norm}} < T_i^{(5\%)} \quad \text{and} \quad \vec{p}_i^{\text{norm}} > T_i^{(5\%)}$$

where $\vec{o}^{\text{norm}}$ is the centroid.

7.4 Rank-normalized Signal Index (RSI)

To assess the local signal strength of expression or shift vectors across the space, Tensor Omics introduces the *Rank-normalized Signal Index (RSI)*. This index provides a scale-independent estimate of the absolute magnitude of a vector, contextualized within the empirical distribution of magnitudes in the current field.

Given a set of vectors

$$\mathcal{V} = \{\vec{v}_1, \dots, \vec{v}_N\} \subset \mathbb{R}^n$$

each vector's Euclidean norm is computed:

$$s_i = \|\vec{v}_i\|$$

These norms are assembled into a global magnitude distribution. To reduce the impact of noise and outliers, we extract the empirical 5th and 95th percentiles:

$$Q_5 = \text{Percentile}_{5\%}(\{s_i\}), \quad Q_{95} = \text{Percentile}_{95\%}(\{s_i\})$$

For each vector, the RSI is then computed as a clipped, percentile-normalized score:

$$\text{RSI}_i = \min \left(1, \max \left(0, \frac{s_i - Q_5}{Q_{95} - Q_5} \right) \right)$$

This formulation maps raw magnitudes into the unit interval $[0, 1]$, where values near 0 indicate weak or noise-like vectors, and values near 1 indicate vectors with strong signal relative to the field. Clipping ensures robustness even in the presence of heavy-tailed or multimodal distributions.

Note: RSI applies uniformly to expression fields and shift fields. In the context of signal-based subfield detection, it provides a direct and empirically grounded method for excluding noise vectors and prioritizing high-signal regions. In future extensions, RSI may also be computed on scalar fields derived from second-order tensors (e.g., Frobenius norm of surface tensors).

7.5 Relative Frobenius Index (RFI)

For any set of d vectors $\{\vec{v}_1, \dots, \vec{v}_n\}$, e.g. gene families, populations, or genes belonging to the same biological process or molecular function group, the vectors are assembled as columns in a matrix $M \in \mathbb{R}^{n \times m}$. The signal strength of this vector-group (matrix) is computed as:

$$\text{RFI} = \frac{\|M\|_F}{\sqrt{m}} = \frac{\sqrt{\sum_{i=1}^n \sum_{j=1}^m M_{ij}^2}}{\sqrt{m}}$$

This provides a normalized estimate of signal per vector and is used to assess subfield strength.

Note: The Frobenius norm is here used as a scalar estimate of the total signal energy contributed by a set of expression vectors. The matrix $M \in \mathbb{R}^{n \times m}$ contains m gene vectors (columns) embedded in a fixed n -dimensional semantic space. The RFI, defined as $\|M\|_F / \sqrt{m}$, reflects the average per-vector magnitude in a root-mean-square sense. This formulation emulates the cumulative effect of a gene set on a test particle exposed to their additive influence, without requiring square or self-adjoint structure. It supports comparison of sets with arbitrary cardinality m , all embedded in a shared space $\mathbb{R}^n$.

All empirical distributions are stored in Fortran arrays and used to assign significance thresholds for downstream classification and visualization.

8 Outlier Detection and Shift Vector Calculation

After constructing the vector space and identifying gene families, Tensor Omics detects divergent expression profiles within each family by comparing individual expression vectors to their respective family centroid.

Let $\vec{o} \in \mathbb{R}^n$ denote the centroid of a given family (typically the mean ortholog expression vector), and let $\vec{p}$ denote the expression vector of a candidate paralog. The *raw shift distance* between $\vec{p}$ and the centroid is defined as:

$$d_{po} = \|\vec{p} - \vec{o}\|$$

To determine whether this shift is significant, we normalize the raw distance using the maximum intra-family distance from the centroid. Let $\vec{f}_i$ denote all other expression vectors in the same family. The *Relative Distance Index (RDI)* is calculated as:

$$\text{RDI}_{po} = \frac{d_{po}}{\max_i \|\vec{f}_i - \vec{o}\|}$$

Vectors with an RDI greater than the 95th percentile of the empirical RDI distribution (pooled across all families) are flagged as significant outliers.

Important: Only the raw shift distance d_{po} is used for downstream geometric interpretation, such as vector field construction, trajectory analysis, and clock plot projection. The normalized RDI is used solely for filtering purposes, to robustly identify biologically meaningful divergence across gene families.

Identified outliers are stored as index references in a dedicated Fortran array. For each significant outlier, a corresponding *shift vector* is calculated:

$$\vec{s}_{po} = \vec{p} - \vec{o}$$

These shift vectors are added to the central vector space store ('vec_store') and indexed in the relevant K-D Trees (see section 4.1).

9 Tissue Versatility Calculation

Cosine of the angle θ between an expression vector and the space diagonal constant ($\vec{d}$):

$$\cos(\theta) = \frac{\vec{p} \cdot \vec{d}}{\|\vec{p}\| \cdot \|\vec{d}\|}$$

where $\vec{d} = (1, 1, \dots, 1)$ is the space diagonal and a constant.

This measure informs about a gene expression tissue (axis) specificity, or versatility, respectively. The larger θ the more the closer an expression vector's localization to one of the space's axes is. Thus its expression is more specific to the tissue represented by that axis. Conversely, the smaller θ the closer the expression vectors localization to tissue uniform expression along the space diagonal $\vec{d}$ and thus the more versatile the gene's expression in the tissues (axes).

10 Clock Plot Projection and Rotation Angle Computation

- All shift vectors are projected into the **Relative Axes Plane (RAP)**, orthogonal to the space diagonal and intersecting the origin.
- The following quantities are calculated for each outlier:
 - Projected shift vector in RAP.
 - Normalized (unit-length) projection.
 - Clock hand vector from origin to projected point.
 - Rotation angle (standardized as clockwise) between projected centroid and outlier.
- Results are stored in a structured Fortran matrix **rap_store**.
 - For each outlier i , store:
 - * Rows 1 to n : $\vec{s}_i^{\text{RAP}}$
 - * Rows $n + 1$ to $2n$: $\vec{s}^{\text{RAP, norm}}$
 - * Rows $2n + 1$ to $3n$: $\vec{s}_i^{\text{clock}}$
 - * Row $3n + 1$: rotation angle in radians

10.1 Efficient Vector Lookup Using K-D Trees

To enable fast and scalable querying of gene expression vectors, Tensor Omics employs K-D Trees—an efficient data structure for multidimensional nearest-neighbor searches. The trees are constructed at the very end of the pipeline. Four trees in total, two for all expression and shift vectors, where both are contained in **vec_store** (see 4), and two further for the RAP vectors in **rap_store** (see 10).

Two distinct trees are constructed:

- **cart_tree**: A K-D Tree built from the raw (unnormalized) expression vectors. It supports neighborhood searches in the original Euclidean space, useful for detecting local density patterns or spatial coherence among genes.

- **sphere_tree:** A second K-D Tree constructed from unit-length normalized vectors. This tree enables fast lookup of vectors with similar direction, allowing angular queries based on cosine similarity.

These lookup structures are only built during the final preprocessing stage, prior to interactive analysis. This deferred construction minimizes computational overhead during earlier pipeline steps and ensures consistency with the final state of the vector space. Together, the two trees provide complementary geometric access to the expression landscape.

11 Geometric Tools for Semantic Interpretation

Tensor Omics provides a suite of geometric tools—referred to as *semantic geometry measures*—that support interpretation, subfield detection, and comparative analysis of expression vector fields. These tools operate directly on the vector space and are designed to align with intuitive biological hypotheses.

11.1 Direction-Based Subfield Collector

To detect expression subfields aligned with a biologically meaningful trend, the user defines a direction-of-interest (DOI) vector $\vec{v}_{\text{doi}} \in \mathbb{R}^n$. This may correspond to, for example, increased healing rate, higher oil content, or a linear combination of expression axes.

For each expression vector $\vec{v}_i$, the cosine similarity with the DOI is computed. The method supports two modes, controlled by a Boolean input argument:

- **Normalization enabled:** both vectors are normalized to unit length, and the cosine is computed as:

$$\cos(\theta_i) = \vec{v}_i^{\text{norm}} \cdot \vec{v}_{\text{doi}}^{\text{norm}}$$

This mode emphasizes *direction* over magnitude and is suited for analyses where only the semantic trend matters, such as expression coordination in transcription factors or between tissues.

- **Normalization disabled:** the cosine is computed relative to the magnitude of the expression vector:

$$\cos(\theta_i) = \frac{\vec{v}_i \cdot \vec{v}_{\text{doi}}^{\text{norm}}}{\|\vec{v}_i\|}$$

This mode retains the influence of vector magnitude and is recommended when working in fold-change spaces (e.g., stress responses), where high expression shifts carry biological meaning.

The resulting cosine values $\cos(\theta_i)$ are used to construct an empirical distribution. Vectors with the strongest projection onto the DOI (e.g., top 5% by cosine value) are selected as a directional subfield. These can then be characterized using the Subfield Descriptor (Section ??).

11.1.1 When to Normalize (Direction-Only Interpretation)

Normalization is recommended when:

- The biological question relates to the **direction of change** in expression across conditions.
- Expression magnitude is not meaningful (e.g., low-expression transcription factors).
- Comparing semantic shifts across gene families or tissues.

11.1.2 When Not to Normalize (Magnitude Matters)

Omitting normalization is appropriate when:

- The space encodes **magnitude changes**, e.g., fold-change after stress.
- High-magnitude responses are biologically meaningful, such as upregulation in immune response genes.
- The DOI is used to detect strong response contributors in a specific context.

11.1.3 Selection of Directional Subfield

The cosine values $\cos(\theta_i)$ are collected across all vectors and an empirical distribution is formed. The top 5% (or another percentile threshold) are selected as the directional subfield—those vectors with the strongest alignment (largest projection) onto the DOI.

The selected subset is returned as a list of vector indices and can be further analyzed using subfield descriptors (Section ??).

11.2 Signal-Based Subfield Detection

In addition to metadata-driven and direction-of-interest-based methods for subfield collection, Tensor Omics supports a purely signal-based approach for identifying coherent expression or shift regions. This method—referred to as **Signal-Based Subfield Detection**—operates solely on vector magnitude and can be applied to both expression and shift fields. It provides a simple yet robust means of discovering compact, high-activity modules without assumptions about directionality or coherence, and serves as an efficient precursor to tensor field analysis.

11.2.1 Vector Collection

The procedure begins by selecting the tensor field to be analyzed:

- the **expression field**, consisting of gene expression vectors $\vec{p} \in \mathbb{R}^n$, or
- the **shift field**, with shift vectors defined as $\vec{s} = \vec{p} - \vec{o}$, where $\vec{o}$ is the gene family’s centroid.

The full set of vectors is assembled as:

$$\mathcal{V} = \{\vec{v}_1, \dots, \vec{v}_N\} \subset \mathbb{R}^n$$

Each $\vec{v}_i$ is selected from one or more of the following:

- Original expression or shift vectors (raw),
- Interpolated vectors at grid points,
- Smoothed vectors obtained from post-interpolation Gaussian smoothing.

For each vector $\vec{v}_i$, the signal strength measure depends on the chosen field:

- In the **expression field**, signal strength is evaluated using the Rank-normalized Signal Index (RSI), as described in Section 7.4. This ensures percentile-based normalization across diverse signal ranges.
- In the **shift field**, signal strength is computed directly via the Euclidean norm $\|\vec{s}_i\|$, which reflects absolute expression change relative to the centroid.

This modular design enables application across multiple resolution levels, from raw data to interpolated or smoothed fields. While the default implementation targets interpolated shift vectors, the framework generalizes to any well-defined vector field, including future fields derived from second-order tensors.

Note: In future extensions, this method may be applied to scalar fields derived from surface or covariance tensors (e.g., Frobenius norm of second-order structures). In such cases, the same region detection logic can be applied to identify coherent high-curvature or high-rotation zones.

11.2.2 Magnitude Distribution and Noise Filtering

To isolate signal-dense regions, the Euclidean norms $\|\vec{v}_i\|$ of all vectors in $\mathcal{V}$ are computed. An empirical noise threshold is then derived by estimating the lower 5th percentile:

$$\tau = \text{Percentile}_{5\%} \left(\{\|\vec{v}_i\|\}_{i=1}^N \right)$$

Vectors with norm less than τ are considered sub-threshold and excluded from further region growth.

11.2.3 Region Growing

From each unmarked vector $\vec{v}_j$ with $\|\vec{v}_j\| \geq \tau$, a local subfield is grown radially:

- Initialize the region with the seed $\vec{v}_j$ at radius $r = 0$.
- Increment the radius stepwise, adding unmarked neighboring vectors within radius r , provided they meet the magnitude threshold τ .
- If a full radius expansion includes only sub-threshold vectors, the region terminates.

All spatial distances are computed in the native semantic space $\mathbb{R}^n$ using Euclidean distance. Grid vectors and original data vectors may both serve as candidates in the same collection, depending on analysis scope.

11.2.4 Region Marking and Continuation

All vectors added to a region are marked. The procedure continues iteratively over the remaining unmarked, non-noise vectors until no new regions can be seeded.

11.2.5 Interpretation

This method segments the vector field into coherent *signal-rich subfields* based solely on local vector magnitude. In the expression field, the resulting regions may reflect domains of elevated transcriptional activity. In the shift field, they may reveal adaptation zones, such as tissue-specific neofunctionalization or spatially localized responses to perturbation. Because no directionality or covariance modeling is involved, this method is resilient to noise and sample sparsity.

Note: The threshold τ may be adjusted (e.g., 1%, 10%) to modulate sensitivity. The procedure generalizes naturally to higher-order tensors by applying the same algorithm to scalar-valued summary fields (e.g., Frobenius norms of second-order tensors).

11.3 Workbench for Interactive Exploration

The Tensor Omics workbench is a lightweight, browser-based environment for exploring the expression space and its geometric structures. It supports:

- 2D and 3D projections (e.g., RAP plane, PCA), with zoom and rotation.
- Efficient filtering via metadata or geometric queries using K-D Trees.
- Visual encoding of vector properties (color, shape, size) by metadata attributes.
- Highlighting of gene families, pathways, or other annotated subsets.

Plots are exportable (e.g., to SVG or PDF), and the current view state can be serialized and restored for reproducibility e.g. by attaching it as request parameters and thus enabling linking to preconfigured interactive plots. This supports sharing, publication, and reproducible workflows.

12 Time Series Analysis

Tensor Omics applies a two-stage process to prepare time series expression trajectories for vector field analysis. This approach addresses the common problem of sparsely sampled or unevenly spaced time points in transcriptomic time courses and ensures that divergence detection remains sensitive and robust across genes and families.

12.1 Stage 1: Per-Gene Trajectory Interpolation

For each gene, we construct an interpolated expression trajectory using Gaussian kernel-based interpolation. Let the original time points be $t_1, t_2, \dots, t_n$, with corresponding expression vectors $\vec{e}_{t_i} \in \mathbb{R}^d$. These are interpolated to a denser trajectory with $T = \max(10n, 30)$ total time points, equally spaced over the original time interval.

Interpolation is performed independently per gene, without considering other genes. This preserves the native trajectory shape while increasing temporal resolution. For each interpolated time point t , a Gaussian kernel $K(t, t_i)$ is applied over all original points:

$$K(t, t_i) = \exp\left(-\frac{(t - t_i)^2}{2\sigma_i^2}\right)$$

where σ_i is the local kernel width, chosen adaptively based on sampling density (see below).

12.2 Stage 2: Contextual Smoothing Across Gene Families

To reduce noise and incorporate contextual biological similarity, each interpolated trajectory is smoothed using a Gaussian kernel over time, borrowing support from orthologs and paralogs.

Each smoothed time point $\vec{e}_t$ is computed as a weighted average of temporally adjacent points from:

- The same gene (denoted as G)
- Orthologs within the family (denoted as O)
- Paralogs within the family (denoted as P)

The total contextual weights are distributed as:

$$\frac{6}{9} \text{ for } G, \quad \frac{2}{9} \text{ for } O, \quad \frac{1}{9} \text{ for } P$$

Within each group, weight is evenly divided and modulated by a temporal kernel:

$$w_{gi} = \frac{6}{9|G|}, \quad w_{oi} = \frac{2}{9|O|}, \quad w_{pi} = \frac{1}{9|P|}$$

The Gaussian weight based on temporal distance is:

$$K(t, t_i) = \exp\left(-\frac{(t - t_i)^2}{2\sigma_i^2}\right)$$

The final smoothed vector is:

$$\vec{e}_t = \frac{\sum_i w_i K(t, t_i) \vec{e}_{t_i}}{\sum_i w_i K(t, t_i)}$$

12.3 Adaptive Kernel Width

The kernel width σ_i adapts to local sampling density:

$$\sigma_i = \begin{cases} t_2 - t_1 & \text{if } i = 1 \\ \frac{1}{2}(t_{i+1} - t_{i-1}) & \text{if } 1 < i < n \\ t_n - t_{n-1} & \text{if } i = n \end{cases}$$

12.4 Shift Vector Construction

After interpolation and smoothing, the trajectory for each gene is now defined as a continuous series of vectors $\vec{p}_t$. For any gene family, the centroid trajectory $\vec{o}_t$ is computed as the mean expression trajectory of all orthologs. The shift vector at each time point is then:

$$\vec{s}_t = \vec{p}_t - \vec{o}_t$$

These shift vectors form the time-dependent shift field used for downstream divergence and angle-based analyses.

12.5 Angular Projection and Clock Plot Modes

For each time point t , we estimate the forward velocity of the gene’s expression trajectory as:

$$\vec{v}_t = \vec{p}_{t+1} - \vec{p}_t$$

From this velocity vector, we define a local projection plane Π_t orthogonal to $\vec{v}_t$. Into this plane we project:

- the shift vector $\vec{s}_t = \vec{p}_t - \vec{o}_t$
- the anchor axis (AA), as a fixed reference direction in $\mathbb{R}^d$

The clockwise angle between these projected vectors forms the tissue preference rotation at time t , which is visualized in the time trajectory clock plot.

12.6 Clock Plot Visualization Modes

We define two modes for clock plot visualization based on the angular coherence of local velocity vectors $\vec{v}_t$:

Mode A (Low Angular Variance)

If the angular variance of the velocity vectors $\vec{v}_t$ is below a threshold, we compute the mean velocity vector $\vec{v}_{\text{mean}}$, derive a single global projection plane Π_{mean} , and use this for all projections. This yields an interpretable, consistent visualization with a coherent coordinate system origin and visible angle to the space diagonal.

Mode B (High Angular Variance)

If angular variance is high, each time point is projected into its own local plane Π_t . These planes are stacked in the final clock plot by superimposing their origins. The anchor axis and shift vector are rendered as if co-planar. This introduces spatial distortion but preserves directional interpretation at each time point. A supplementary plot indicates angular variance over time.

12.7 Angular Shift Velocity (ASV)

In Mode B, we additionally compute the angular velocity of shift direction over time. At each time point t , the ASV is:

$$\text{ASV}_t = \angle(\vec{s}_{t+1}^\perp, \vec{s}_t^\perp)$$

where $\vec{s}_t^\perp$ denotes the projection of the shift vector into Π_t . This metric captures turning behavior in shift trajectories and may reveal biological transitions or inflection points in tissue preference.

12.8 Mjöltnir’s Handle — Canonical Axis Reconstruction for Eitri Surface Extraction

- **Purpose:**
 - Defines a unified, smoothed tracing axis for surface reconstruction within a semantic subfield.
 - Replaces inconsistent per-axis Brokk cylinders with a continuous, empirically derived semantic rail.
 - Anchors all surface geometry: clock hands, pie slicing, and surface candidates.
- **Step 1 — Extract Brokk Axes:**
 - For expression fields: apply PCA to normalized vectors to get spread axes.
 - For shift fields: compute covariance matrix of shift vectors; first eigenvector is dominant flow.
 - Retain 1–3 eigenvectors if needed; typically use the first (dominant) as $\vec{b}$.

- **Step 2 — Compute Clock Hand Projections:**

- For each vector $\vec{v}_i$ in the subfield, decompose:

$$\vec{v}_i = \vec{r}_i + \vec{v}_i^{\text{clock}}, \quad \text{where} \quad \vec{r}_i = \langle \vec{v}_i, \vec{b} \rangle \cdot \vec{b}, \quad \vec{v}_i^{\text{clock}} = \vec{v}_i - \vec{r}_i$$

- The root vectors $\vec{r}_i$ define a rough point cloud aligned with the subfield’s dominant axis.

- **Step 3 — Smooth Root Vector Cloud:**

- Apply Gaussian kernel smoothing over the set of $\vec{r}_i$.
- This produces a smooth curve through semantic space — the **Mjöltnir handle**.
- Recommended kernel width:

$$\sigma = 1.2 \cdot d_{\text{grid step}}, \quad \text{truncated at } 3\sigma$$

- Adaptive σ may be used based on local density.

- **Step 4 — Surface Extraction via Eitri 2.0:**

- Divide Mjöltnir’s handle into equal-length or quantile-based segments.
- For each segment:
 - * Construct a plane orthogonal to the handle (TRAP).
 - * Form a cylindrical shell (tube slice) around the segment.
 - * Project vectors into the TRAP and perform angular binning.
 - * In each pie slice, select the vector with maximal radial length as a surface candidate.
- Optional smoothing across overlapping segments yields continuous surfaces.

- **Interpretation:**

- Mjöltnir’s handle is the *semantic spine* of the surface — the first structure carved before the geometry is wrapped.
- Enables reversible and modular interpretation of subfield geometry.
- Removes ambiguity from axis choice and enforces directional consistency across surface detection.

12.9 Instantaneous Acceleration (Future Extension)

As a potential extension, we define instantaneous acceleration as:

$$\vec{a}_t = (\vec{p}_{t+1} - \vec{p}_t) - (\vec{p}_t - \vec{p}_{t-1}) = \vec{p}_{t+1} - 2\vec{p}_t + \vec{p}_{t-1}$$

This second-order derivative could be used to detect abrupt transitions in expression dynamics, e.g., rapid shifts in cell state or bifurcation points in developmental or stress trajectories. It is currently not used in visualization but retained for future interpretability analyses.

12.10 Biological Interpretation

This time-aware geometric framework allows Tensor Omics to identify and interpret dynamic expression shifts. Local angular trajectories provide insight into progressive tissue reprogramming, while metrics such as ASV capture the smoothness or abruptness of functional divergence over time. The projection modes ensure interpretability across both smooth and complex expression paths, enabling application across diverse systems from developmental timelines to stress response series.

12.11 Implementation Notes

All trajectory interpolation, smoothing, projection, and angle computations are performed in Fortran using preallocated arrays for performance and reproducibility. Projection operations avoid

13 Gilgamesh Learning: Semantic Field-Aware Pattern Classification

13.1 Overview

The **Gilgamesh Learning Layer** is the first structured machine learning stage in Tensor Omics. It performs explainable, geometric pattern classification over semantically aligned vector fields, such as expression vectors or shift vectors, using robust aggregation of canonical field descriptors across a normalized subfield space. The method is fully deterministic, aligned with F42-compliant memory handling, and compatible with both supervised and unsupervised classification (e.g., SVMs or interpretable logistic regression).

13.2 Motivation and Biological Context

Traditional ML pipelines in omics use raw gene-level input or PCA-transformed data. These approaches:

- Do not respect semantic coherence of expression vector fields.
- Obfuscate spatial locality or directional geometry.
- Often ignore explainability in favor of brute predictive power.

In contrast, **Gilgamesh Learning** leverages the latent geometry of semantic fields constructed in TOX to provide biologically interpretable classification, tracing the activation of regions (cylinder bins) and their descriptors, and linking learned patterns back to cellular processes or clinical labels.

13.3 Method

Let $\mathcal{F} \subset \mathbb{R}^n$ be a subfield of expression or shift vectors, constructed post-normalization, smoothing, and grid interpolation. Let H denote the **Mjölfnir handle**, i.e., the primary flow axis of the subfield, computed via kernel-smoothing over Brokk root vectors as described in the corresponding section.

13.3.1 Cylindrical Binning Along Mjölfnir’s Handle

1. Compute the set of **Brokk root vectors** by projecting each expression or shift vector onto H and recording the foot of the perpendicular dropped from each vector onto the handle. These form a dense sampling of trajectory-aligned coordinates.
2. Partition the handle H into g equal-sized bins based on the distribution of Brokk root vector coordinates, typically using empirical quantiles (e.g., five-percentile bins).
3. Around each segment of H , construct a cylindrical region $\mathcal{C}_i$ that captures spatially co-local vectors.
4. Assign each vector in $\mathcal{F}$ to its corresponding $\mathcal{C}_i$ using proximity to the associated Brokk root coordinate.

Directional Alignment Convention: To ensure consistent orientation, define "upward" along the handle as the direction in which the cosine between the global principal flow vector and the linearized Mjölfnir handle is positive. If negative, reverse the order of bins. This ensures that all bin sequences respect a canonical semantic up-down structure across experiments and fields.

13.3.2 Canonical Descriptors

For each bin $\mathcal{C}_i$, construct a matrix M_i from collected vectors (as columns). Extract the following descriptors:

- **Signal Strength** $s_i = \|M_i\|_F$ (Frobenius norm)
- **Directional Spread** $r_i = \text{rank}(M_i)$
- **Flow Vector** $\vec{f}_i$: First eigenvector of $M_i M_i^\top$

- **Rotational Moment** $\vec{\rho}_i$: Skew-symmetric component of $M_i M_i^\top$

Construct a **concatenated feature vector**:

$$\vec{x} = \text{concat}(s_1, r_1, \vec{f}_1, \vec{\rho}_1, \dots, s_g, r_g, \vec{f}_g, \vec{\rho}_g)$$

This $\vec{x}$ represents the full subfield $\mathcal{F}$ in structured geometric terms.

13.3.3 Training and Classification

Train a supervised classifier (e.g., linear SVM) using vectors $\vec{x}$ from labeled subfields. For a trained SVM with weights $\vec{w}$ and bias b , the decision function is:

$$\text{score}(\vec{x}) = \vec{w}^\top \vec{x} + b$$

13.4 Explainability

The contribution of each bin-descriptor pair can be extracted from $\vec{w}$:

- Rank the components $x_i \cdot w_i$ in descending order.
- Use cumulative sum until 80% of $\text{score}(\vec{x})$ is explained.
- Associate top-contributing bins back to their positions in H .

This yields:

- Spatially localized regions of activation.
- Descriptor-level explanations (e.g., flow, rotation).
- Optional lexical/ontological annotations via linked metadata (cf. GAWD).

13.5 Semantic Alignment and Orientation of Subfields

To enable comparability and structured learning across subfields, a consistent orientation convention is required. This is achieved via a canonical alignment using the subfield’s main flow axis and the global semantic spine.

Let H denote the Mjöltnir handle, constructed as a smoothed curve through Brokk root vectors (the perpendicular foot points of vectors projected onto the main axis). Let $\vec{v}_{\text{flow}}$ be the first eigenvector of the covariance matrix $C = MM^\top$ for all vectors in the subfield.

We define:

$$\text{orientation sign} = \text{sign}(\vec{v}_{\text{flow}} \cdot \vec{v}_H)$$

where $\vec{v}_H$ is the unit vector pointing from the start to the end of the linearized handle H .

- If orientation sign > 0 , the handle is already aligned "upward" (biologically or semantically increasing).
- If orientation sign < 0 , the bin sequence is reversed to restore canonical alignment.

This alignment step ensures that all subfields are sliced and encoded in a directionally consistent fashion, crucial for both classification and downstream glyph interpretation. It also preserves directional semantics across layers (e.g., Gilgamesh $\rightarrow$ Nubecita) and across different samples or biological conditions.

13.6 Glyph Description via Lexical Metadata

After training, apply the SVM model to each of the original training subfields. Compute the decision score for each. Extract the subfields falling into the top 5th percentile of decision scores. For each such activated subfield $\mathcal{F}_i$, compute its mean lexical vector $\vec{\ell}_i$ from associated metadata (e.g., Gene Ontology, KEGG, Mapman, patient notes).

The resulting set $\{\vec{\ell}_1, \dots, \vec{\ell}_k\}$ forms the ontological glyph description of the learned class. These mean vectors can be further clustered, projected, or plotted to study the coherence or diversity of the learned concept. By default, all such $\vec{\ell}_i$ are retained.

13.7 Biological Applications

- Identify class-specific tissue shifts (e.g., cancer vs. control).
- Trace developmental trajectories (e.g., stage-specific expression flow).
- Localize divergent processes across species in comparative studies.
- Link clinical outcomes to geometric shifts in expression fields.
- Extract data-native glyphs summarizing learned biological programs.

13.8 Advantages Over Conventional ML

- Fully explainable: input dimensions are geometric and interpretable.
- Robust to noise: uses smoothed field descriptors, not raw vectors.
- Compatible with GAWD and Eitri for recursive analysis.
- Aligned with TOX principles: no imposed parametric model, minimal transformation.
- Empowers symbolic downstream use: enables second-layer learning (Nubecita).

13.9 Example Pseudocode (Sketch)

```
function gilgamesh_vector(subfield, mjolnir_handle, n_bins):
    bins = split_handle_into_bins(mjolnir_handle, n_bins)
    result = []
    for bin in bins:
        vectors = collect_vectors_in_cylinder(subfield, bin)
        M = form_matrix_from_vectors(vectors)
        s = frobenius_norm(M)
        r = matrix_rank(M)
        f = first_eigenvector(M * transpose(M))
        rho = rotation_vector(skew(M * transpose(M)))
        result.append([s, r, f, rho])
    return concatenate(result)
```

14 Shape truthful clustering

14.1 Summary

- Use spherical kernels at each grid point to measure **vector density** (count of vectors inside radius r).
- Label each point with this count.
- Sort descending.
- Start growth from the densest point.
- Iteratively grow the sphere, aggregating nearby high-density spheres until **running density derivative drops** (signal plateau).
- In shift fields, alternatively use **cones** to study whether directional flows converge toward dense regions. This addresses directional accumulation and potential biological attractors.

14.2 Interpretation by Vector Type

14.2.0.1 Expression vector fields (use spheres):

- Best suited for detecting **isotropic clustering** in high-dimensional semantic expression space.
- Reveals:
 - **Conserved regulatory hubs**
 - **Functionally coherent modules**
 - **Developmental plateaus or tissue convergence attractors**
 - **Cell-type specific modules** conserved across samples or species

14.2.0.2 Shift vector fields (use cones):

- Best suited for tracing **directional convergence** of functional shifts.
- Reveals:
 - **Strong directional pull** from paralog groups or neofunctionalizing subsets
 - **Convergent evolutionary innovations**
 - **Stable transcriptomic transitions** across conditions or species
- Use of cones can identify whether directional flow leads into dense attractor zones.

14.3 Verdict

Highly promising. Integrate as a “Bobito-density mode” (working name: **Kolob**), and pin as a method for **density-informed seed detection** in both expression and shift fields.

14.4 Note

While spheres are the natural geometry for isotropic density detection, cones are preferable in *shift vector fields* where direction matters. Cones reveal whether vector flows converge into dense subregions, offering geometric insight into evolutionary or functional attractor zones.

15 Detection of Subfunctionalization and Dosage Effects via Dynamic Programming

This section formalizes the algorithmic strategies to identify subfunctionalization and dosage effect patterns among in-paralogous gene sets, based on semantic expression vectors in Tensor Omics. The procedures leverage dynamic programming and bitmask representations to achieve efficient, branch-pruned exploration of possible paralog subsets.

15.1 Overview

Given:

- The ancestral ortholog expression vector $\vec{o}$.
- A set of n in-paralog expression vectors $\vec{p}_1, \vec{p}_2, \dots, \vec{p}_n$, stored as columns of a Fortran 2D array.

The goal is to identify subsets of paralogs whose summed expression vectors approximate $\vec{o}$, within a threshold defined as the lower five percentile of family centroid distances (RDI threshold). Two detection scenarios are supported:

1. **Subfunctionalization:** The paralogs exhibit significant angles to the ortholog, partitioning ancestral expression.
2. **Dosage effect:** The paralogs have small angles to the ortholog and sum to a magnitude significantly exceeding $\|\vec{o}\|$.

15.2 Data Structures

- All subset membership is encoded as bitmasks of the ranks of the paralog indices, stored as integer values.
- A dynamic programming storage array M holds the active subsets to be extended at each iteration, also represented as bitmasks.
- An array of norms for each paralog, sorted ascending, is maintained for pruning decisions.
- A logical vector marks which paralogs remain candidates at each step.

15.3 Algorithm Steps

15.3.0.1 Initialization

- For all single paralogs, measure their angle to $\vec{o}$.
- Apply angle threshold filters:
 - Subfunctionalization: only paralogs with angles greater than the gene-family median angle are marked active.
 - Dosage effect: only paralogs with angles below the gene-family median or lower five percentile are marked active.
- Generate all pairs ($k=2$) of active paralogs to begin.

15.3.0.2 Dynamic Programming Iteration For each iteration k :

1. Iterate over the bitmask-encoded subsets of size $k - 1$ stored in M .
2. For each such subset:
 - (a) Extend the subset by adding one more paralog, using bitwise operations to set zero bits to one.
 - (b) Compute the sum of the expression vectors of the extended subset.
 - (c) Compute the residual:

$$\vec{r} = \vec{o} - \sum_{i \in S} \vec{p}_i$$

- (d) Test whether:

$$\|\vec{r}\| < \text{RDI threshold}$$

If true, store this bitmask in the results array and *do not* pass it to the next iteration.

- (e) Otherwise, apply pruning logic:

- (f) For *subfunctionalization*:

- Check the minimal length of any still-active paralog:

$$\min_i \|\vec{p}_i\| < \|\vec{r}\|$$

If this fails, prune the subset as further extension is impossible.

- (g) For *dosage effect*:

- For dosage effect, measure the angle of the current subset sum to $\vec{o}$ and prune if this exceeds the acceptable bound.

- (h) If the subset is marked to continue, store its bitmask in M for the next iteration.

3. Increment k and repeat until k exceeds the maximum paralog search size or no active subsets remain.

15.3.0.3 Final Filtering At the end of the dynamic program, all positive hits are stored as bitmask-encoded subsets. These represent:

- Subfunctionalized paralog subsets whose summed expression reconstructs $\vec{o}$ within the RDI threshold.
- Dosage-effect subsets whose sum is aligned with $\vec{o}$ but exceeds its magnitude.

15.4 Notes on Implementation

- Subset bitmasks can be efficiently enumerated by setting zero bits in rank order to one.
- Final validation should always be performed using the RDI threshold, regardless of pruning.
- A global maximum paralog subset size (e.g., 15) is recommended to control complexity.
- For large families, consider pre-filtering based on tissue specificity, expression variance, or percentile angles.

15.5 Pseudocode

```
Initialize:
  results = []
  active_sets = all bitmask pairs from angle-filtered paralogs

for k = 2 to MAX_PARALOGS:
  next_active_sets = []
  for each bitmask in active_sets:
    for each still-active paralog not in bitmask:
      candidate_mask = bitmask with paralog bit set
      sum_vector = sum of paralogs in candidate_mask
      residual = o - sum_vector
      if norm(residual) < threshold:
        store candidate_mask in results
      else:
        if dosage:
          if angle(sum_vector, o) too large:
            continue
          if min(paralog norms) >= norm(residual):
            continue
        next_active_sets.append(candidate_mask)
  active_sets = next_active_sets
```

This algorithm achieves robust and scalable detection of subfunctionalization and dosage patterns by systematic, bitmask-driven dynamic programming with domain-motivated pruning rules.

16 Trajectory-Based Signed Contribution Analysis

16.1 World Bank Example Dataset

To illustrate the methodology, we consider a simplified World Bank style dataset comprising socio-economic indicators collected annually over $T = 21$ years for approximately 200 countries. Each year is represented by a table whose rows correspond to countries and whose columns correspond to quantitative indicators. By stacking the yearly tables in temporal order, each country forms a trajectory

$$\vec{p}_1, \vec{p}_2, \dots, \vec{p}_{21},$$

that is, a sequence of vectors moving through a semantic space in which each axis represents a specific measurement.

Typical indicators include female and male illiteracy rates, female and male income, mortality rates, unemployment levels, vaccination coverage, and per-capita GDP. These indicators serve as **factors** in our analysis, describing socio-economic conditions that might influence selected outcomes. Additional indicators, such as the proportion of women over 60 or composite gender-equality indices, act as **dependent variables**. Together, these measurements provide a high-dimensional representation of each country’s temporal evolution, making the dataset well suited for trajectory-based signed contribution analysis as described above.

16.2 Trajectory related scientific questions

In order to analyse which socio-economic factors most strongly contribute to improvements or deteriorations in selected dependent variables over time, we consider each country as a trajectory in a semantic vector space. Each axis of this space represents one measured indicator, for example female illiteracy, male income, female mortality rate, unemployment, or per-capita GDP. This representation preserves semantic interpretability, since every dimension corresponds to a concrete and meaningful quantity. For each country we observe a sequence of yearly vectors

$$\vec{p}_t = (x_{t,1}, x_{t,2}, \dots, x_{t,n}), \quad t = 1, \dots, T,$$

where T denotes the number of temporal measurements (e.g. 21 years). This formulation allows us to address two central scientific questions: **(i)** which factors contribute most strongly to the evolution of a dependent variable over the entire trajectory, and **(ii)** at which specific time points do exceptional or anomalous contributions occur.

16.3 Covariance-Based Contribution Decomposition

Given a dependent variable D and a factor F , each represented as scalar coordinates of the trajectory vector $\vec{p}_t$, we quantify their instantaneous co-deviation at time t through the signed contribution

$$c_{D,F}(t) = (D_t - b_D) (F_t - b_F),$$

where b_D and b_F denote chosen baselines for the dependent and factor variables. The sign of $c_{D,F}(t)$ encodes directional co-movement: positive contributions indicate that both variables increase simultaneously or decrease simultaneously relative to their baselines, whereas negative contributions indicate divergence. The magnitude reflects the strength of this alignment or misalignment at a specific time point.

Over the full trajectory the total signed contribution is defined as

$$C_T(D, F) = \sum_{t=1}^T c_{D,F}(t).$$

This cumulative measure captures the overall explanatory co-deviation of factor F for dependent D across the entire temporal span.

16.4 Linear Combinations of Factors and Dependents

The geometric formulation immediately generalises to linear combinations of factors and dependents. Let

$$F^* = \sum_{j \in \mathcal{F}} \alpha_j F_j \quad \text{and} \quad D^* = \sum_{k \in \mathcal{D}} \beta_k D_k,$$

where the coefficients α_j and β_k are scientifically motivated weights that reflect clearly articulated hypotheses rather than algorithmic optimisation. The instantaneous contribution between F^* and D^* becomes

$$c_{D^*,F^*}(t) = (D_t^* - b_{D^*}) (F_t^* - b_{F^*}),$$

with total contribution $C_T(D^*, F^*)$ defined analogously. This enables the investigator to formulate hypotheses such as “the combined unemployment and income gap contributes to the gender-equality index” in a mathematically precise and interpretable manner.

16.5 Baseline Modes

Three baseline modes are supported in order to accommodate different scientific aims.

16.5.1 Raw Contributions

In the raw mode we set $b_D = 0$ and $b_F = 0$, analysing the contributions directly on the original scale. This mode is appropriate when deviations from absolute zero are meaningful or when the variables exhibit no strong directional trends.

16.5.2 Minimum-Centred Contributions

For monotonic or growth-oriented variables, such as literacy rates, per-capita GDP, or gender-equality indicators, minimum-centred baselines are often more intuitive. Here we set

$$b_D = \min_t(D_t) \quad \text{and} \quad b_F = \min_t(F_t),$$

so that $D_t - b_D$ and $F_t - b_F$ represent deviations above historically observed minima. This avoids sign inversions that arise when early values lie below the mean and provides a clear interpretation of improvement or deterioration.

16.5.3 Mean-Centred Contributions

For oscillatory or approximately stationary variables it is often favourable to subtract the trajectory mean. In this case

$$b_D = \frac{1}{T} \sum_{t=1}^T D_t \quad \text{and} \quad b_F = \frac{1}{T} \sum_{t=1}^T F_t.$$

Mean centring yields symmetric deviation structure and is appropriate for variables with no strict monotonic trend.

16.6 Optional Inter-Trajectory Normalisation

When comparing contributions across multiple trajectories (for instance, across countries with vastly different magnitudes of income or literacy), it may be advantageous to normalise each trajectory prior to analysis. A natural and robust scaling is

$$v_t^{\text{norm}} = \frac{v_t - \min_k(v_k)}{\max_k(v_k) - \min_k(v_k) + \varepsilon},$$

where ε is a small constant that prevents division by zero. This rescaling maps each trajectory to the interval $[0, 1]$ without altering its shape or directionality. The downstream contribution analysis remains unchanged, because normalisation occurs solely upstream of the contribution computation.

16.7 Permutation Tests

To assess statistical significance for both time-resolved spikes $c_{D,F}(t)$ and total contributions $C_T(D, F)$, we employ permutation tests. The null hypothesis asserts that the factor trajectory and the dependent trajectory are unrelated. Consequently, the temporal structure within each trajectory must be preserved while the pairing between factor and dependent trajectories must be broken. Formally, for each permutation we replace the dependent trajectory D of a given country with the dependent trajectory D^* drawn from a different country, computing the permuted contributions

$$c_{D^*,F}(t) \quad \text{and} \quad C_T(D^*, F).$$

Repeating this procedure generates an empirical null distribution against which observed contributions are compared. Spike-level significance is obtained by comparing individual $c_{D,F}(t)$ values to their null distribution, while trajectory-level significance is based on $C_T(D, F)$.

16.8 Comparisons Between Groups of Trajectories

The above framework extends naturally to group comparisons, for example contrasting contributions between industrialised and developing countries. For group-level inference we compute the empirical distributions of spike values and total contributions for each group. Non-parametric two-sample tests, such as the Wilcoxon rank-sum test, can then be applied to assess systematic differences between groups. Permutation-based group comparisons can also be constructed by permuting group labels under the assumption of exchangeability.

16.9 Visualisation Strategies

Several visualisation modes serve to illustrate the behaviour of contributions. First, distributions of spike values and total trajectory contributions can be shown as histograms or density plots, optionally accompanied by significance thresholds derived from permutation tests. Second, time-resolved spike contributions can be visualised as jittered scatter plots analogous to Manhattan plots, with dotted horizontal lines marking significance cutoffs. Third, factor-dependent summaries can be displayed in two-dimensional scatter plots with axes representing factors and dependents, in which significant trajectories or spikes are highlighted by colour.

16.10 Discussion

A critical reader may raise concerns regarding correlations between factors or between dependents. Although the semantic vector space treats axes as orthogonal by construction, correlations manifest in the empirical positions of the trajectories and therefore appear directly in the contribution values. This framework thus does not eliminate correlation structure; it measures how such structure contributes to co-deviations in a time-resolved manner. A second concern pertains to the choice of baseline. Minimum-centred baselines are particularly suited for growth processes, whereas mean-centred baselines are preferable for oscillatory variables. This flexibility is not a weakness, but ensures that deviations retain their semantic interpretation. Finally, the validity of permutation tests relies on exchangeability across trajectories, a reasonable assumption for many socio-economic analyses but one that can be refined by stratified or region-specific permutation schemes. Overall, the proposed methodology is internally consistent, conceptually transparent, and provides a novel geometric perspective on temporal co-deviation in multivariate systems.

16.11 Velocity- and Acceleration-Based Contribution Analysis

In addition to analysing the trajectories $\vec{p}_t$ themselves, we also examine their discrete temporal derivatives in order to capture the dynamics of change. Throughout this subsection we assume that the time steps are constant (for example, yearly measurements in the World Bank dataset). If time steps were irregular, interpolation would be required prior to the derivative computation, which is not supported in the current implementation.

16.11.1 Velocity Contributions

The discrete velocity vectors are defined as

$$\vec{v}_t = \vec{p}_t - \vec{p}_{t-1}, \quad t = 2, \dots, T.$$

These vectors quantify how the entire socio-economic profile of a country changes from one year to the next. Each component $v_{t,j}$ represents the year-to-year increment or decrement of indicator j , and the magnitude $\|\vec{v}_t\|$ provides a scalar measure of the overall change occurring at time t .

Signed contributions for velocities are defined analogously to ambient contributions. For a dependent variable D and a factor F , and with selected baselines b_D^{vel} and b_F^{vel} , we write

$$c_{D,F}^{\text{vel}}(t) = (v_{t,D} - b_D^{\text{vel}})(v_{t,F} - b_F^{\text{vel}}).$$

The global contribution across the trajectory is then

$$C_T^{\text{vel}}(D, F) = \sum_{t=2}^T c_{D,F}^{\text{vel}}(t).$$

Velocity-based contributions therefore quantify how co-increments or co-decrements of D and F align through time. The largest year-to-year change can be identified by the maximum of $\|\vec{v}_t\|$, marking the point of strongest development or deterioration.

The directional role of individual factors can further be examined by the cosine between the velocity vector and axis F ,

$$\cos(\vec{v}_t, e_F) = \frac{v_{t,F}}{\|\vec{v}_t\|},$$

which describes how much of the total developmental movement at time t is aligned with factor F .

16.11.2 Acceleration Contributions

The discrete acceleration vectors constitute second differences of the trajectory,

$$\vec{a}_t = \vec{v}_t - \vec{v}_{t-1} = \vec{p}_t - 2\vec{p}_{t-1} + \vec{p}_{t-2}, \quad t = 3, \dots, T.$$

Acceleration captures changes in the rate of development. Large values of $\|\vec{a}_t\|$ highlight temporal turning points, shocks, or abrupt transitions in socio-economic behaviour.

Signed contributions for accelerations are defined by

$$c_{D,F}^{\text{acc}}(t) = (a_{t,D} - b_D^{\text{acc}})(a_{t,F} - b_F^{\text{acc}}),$$

with trajectory-level totals

$$C_T^{\text{acc}}(D, F) = \sum_{t=3}^T c_{D,F}^{\text{acc}}(t).$$

Acceleration contributions identify moments where the rate of improvement or deterioration in a factor aligns with the rate of improvement or deterioration in the dependent variable. They therefore highlight joint shocks or inflection points in developmental trajectories.

16.12 RAP-Based Trajectory Contributions

All contribution analyses described above can be performed not only in ambient space but also in the Relative Axes Plane (RAP), where vectors are projected orthogonally to the space diagonal. RAP projection removes the global magnitude component of each trajectory vector, isolating its relative or compositional structure. Formally, letting $\vec{p}_t^{\text{RAP}}$ denote the RAP projection of $\vec{p}_t$, we define

$$\vec{v}_t^{\text{RAP}} = \vec{p}_t^{\text{RAP}} - \vec{p}_{t-1}^{\text{RAP}}, \quad \vec{a}_t^{\text{RAP}} = \vec{v}_t^{\text{RAP}} - \vec{v}_{t-1}^{\text{RAP}},$$

and compute ambient, velocity, and acceleration contributions using $\vec{p}_t^{\text{RAP}}$, $\vec{v}_t^{\text{RAP}}$, and $\vec{a}_t^{\text{RAP}}$, respectively.

While ambient analyses describe absolute improvements or deteriorations, RAP-based analyses characterise relative rebalancing among axes. In the context of the World Bank example, RAP contributions identify whether changes in socio-economic conditions shift the *composition* of development towards or away from gender-related indicators, health axes, or economic axes, independent of overall magnitude changes. This dual perspective of absolute and compositional dynamics provides a more complete geometric characterisation of temporal behaviour in semantic vector spaces.

16.13 Group-Level Trajectory Field Descriptors

In many analyses it is desirable to move beyond individual trajectories and examine the temporal behaviour of entire groups of entities, such as industrialised countries, developing countries, or countries of the global south. In the present context, a group is treated as a collection of trajectories, each of which is observed over the same sequence of time points. Throughout this subsection we assume that the temporal sampling is uniform and that all trajectories share identical time intervals. If time steps were irregular or missing, interpolation would be required prior to constructing group-level descriptors; interpolation is, however, not part of the current implementation.

For each group and each time point t we construct a **local module** by aggregating the trajectory vectors of all group members in a weighted manner. Specifically, we place a Gaussian kernel in time with standard deviation $\sigma = 1$ (the default), which covers a temporal window of approximately three steps to each side. The weight assigned to the vector of country i at time point s is given by

$$w_{t,s} = \exp\left(-\frac{(t-s)^2}{2\sigma^2}\right),$$

and only values with $|t-s| \leq 3$ contribute appreciably when $\sigma = 1$. For a group containing G trajectories, the module at time t therefore consists of the $7G$ vectors from time points $t-3$ to $t+3$, each multiplied by the corresponding weight $w_{t,s}$. We assemble these weighted vectors as columns of the module matrix

$$M_{\text{group},t} = [w_{t,s_1} \vec{p}_{s_1}^{(1)} \quad w_{t,s_2} \vec{p}_{s_2}^{(2)} \quad \cdots \quad w_{t,s_{7G}} \vec{p}_{s_{7G}}^{(G)}],$$

where $\vec{p}_s^{(i)}$ denotes the trajectory vector of country i at time point s .

From each module matrix $M_{\text{group},t}$ we compute the four Tensor Omics field descriptors introduced previously: **(i)** the signal strength (energy), **(ii)** the principal direction of flow, **(iii)** the effective dimensionality (rank), and **(iv)** the rotation vector. The energy is defined as the Frobenius norm

$$E(t) = \|M_{\text{group},t}\|_F,$$

representing the square root of the total signal energy in the module. To obtain the flow and rotational components we first compute the covariance-like matrix

$$C(t) = M_{\text{group},t} M_{\text{group},t}^T.$$

The principal flow direction at time t is the first eigenvector of $C(t)$, denoted $\vec{d}_{\text{flow}}(t)$. The rotational behaviour is obtained from the antisymmetric part

$$A(t) = C(t) - C(t)^T,$$

from which we derive the rotation vector $\vec{r}(t)$. Its direction encodes the axis of rotation, its magnitude the rotation strength, and its sign determines the orientation of rotation. The effective rank of $M_{\text{group},t}$ provides a measure of the module's directional spread or intrinsic dimensionality at time t . See section 17.2 for more details.

To allow direct comparison across indicators with different units and scales, the individual measurement axes are standardised by dividing each axis by its empirical standard deviation prior to constructing the module matrices. With this scaling, the temporal profiles $E(t)$ and $\text{rank}(M_{\text{group},t})$ can be plotted together as two aligned curves: one describing the overall strength of the group's developmental movement and the other describing the complexity or multi-dimensionality of that movement. For example, a high-energy, low-rank regime indicates strong but coordinated group-wide development, whereas a high-energy, high-rank regime reflects strong but heterogeneous changes.

Further geometric insight is obtained by examining the principal flow vectors and rotation vectors in the Relative Axes Plane (RAP). For this purpose, we project $\vec{d}_{\text{flow}}(t)$ and $\vec{r}(t)$ orthogonally to the space diagonal, obtaining $\vec{d}_{\text{flow}}^{\text{RAP}}(t)$ and $\vec{r}^{\text{RAP}}(t)$. These projected vectors can be plotted in a clock-plot representation, in which the axes of the semantic space are arranged at equal angular intervals. The resulting time series of projected flow vectors traces out how the dominant direction of group-level development shifts among the semantic axes, while the series of projected rotation vectors shows how trade-offs or rebalancing dynamics evolve through time.

Taken together, the group-level field descriptors provide a coherent geometric summary of development patterns across multiple entities. They reveal, for example, whether first-world countries undergo smooth, low-rank, high-coordination shifts, whether developing countries experience bursts of high-energy, multi-dimensional change, or whether global-south countries traverse distinct phases in their flow or rotation trajectories. These descriptors also highlight temporal regions of heightened rebalancing, shocks, or directional pivots, thereby offering a compact and interpretable characterisation of group-level dynamics in high-dimensional semantic spaces.

17 Module Descriptors (Tensor-Field Descriptors)

17.1 Concept of Modules (Sets of Vectors) and Vector Fields

In Tensor Omics sets of vectors can inform about scientific questions. E.g. which gene expression vectors are linked to inflammatory response biological processes and what are their characteristics? Or, in socioeconomics which measures like vaccination-rate and highest-education-rate show a similar tendency of increase over the years and is this consistent for all countries?

See section 14 for details on how to find (learn) sets of vectors through shape truthful clustering methods. Note that metadata, i.e. vector labels like the biological process a gene is involved in, can also be used to define sets of vectors. Importantly, two types of vectors and vector sets exist in Tensor Omics: First class ambient vectors and vector fields. First class vectors are plain old vectors that originate at the vector space origin and point to a coordinate. These are e.g. gene expression vectors or literacy-rate. Vector (tensor) fields consist of vectors that have origin coordinates and a vector that indicates the local forces at that coordinate, very much like in magnetic fields or meteorological wind maps. In Tensor Omics those are shift vectors, e.g. the shift of gene expression from the gene family centroid to the paralog gene expression, or changes in literacy-rate between two subsequent years.

Formally, in Tensor Omics, a *vector (tensor) field* consists of vectors together with the coordinates from which they originate,

$$\mathcal{F} = \{ (\vec{x}_i, \vec{v}_i) \mid \vec{x}_i, \vec{v}_i \in \mathbb{R}^n \}.$$

A *field module* is any subset of this field,

$$\mathcal{K} \subseteq \mathcal{F},$$

selected according to biological, semantic, or geometric criteria (e.g. “photosynthesis”, “inflammatory response”, collinearity, or local density).

When the original data consist only of vectors without explicit coordinates, the subfield (*module*) refers simply to a subset of vectors $\{\vec{v}_i\}_{i=1}^m$ extracted from an omics vector space. Such a subset can still be analyzed collectively by arranging its vectors into the matrix

$$M = \begin{bmatrix} | & | & \cdots & | \\ \vec{v}_1 & \vec{v}_2 & & \vec{v}_n \\ | & | & & | \end{bmatrix} \in \mathbb{R}^{d \times n}$$

The cumulative covariance structure of the subfield is then given by

$$C = MM^T,$$

which captures both the magnitude and directional coherence among the vectors. This formulation enables describing the collective influence of the subfield as the cumulative effect exerted on a virtual particle placed conceptually at its center, allowing the use of linear algebraic descriptors to quantify its structure, energy, and dynamics.

17.2 Four Descriptors of a Module / Set of Vectors

To describe the characteristics of a set of vectors (module), be it a module of first class vectors or a vector (tensor) field, we emulate the effect of the vectors onto a virtual particle. To do so, we aggregate the module’s vectors into a Matrix M and create its covariance matrix $C = M \cdot M^T$.

From M and C , we define four key descriptors that characterize the aggregate effect of the subfield on the virtual particle:

17.2.1 Energy (Signal Strength)

$$E = \|M\|_F = \sqrt{\sum_{i=1}^n \sum_{j=1}^m M_{ij}^2},$$

the Frobenius norm of M , measures the total intensity or activity within the module. In gene expression analysis, this reflects the cumulative divergence or activity of genes; in a wind or magnetic field analogy, it corresponds to the total kinetic or magnetic energy in the region.

17.2.2 Rank (Dimensionality or Spread)

$$R = \text{rank}(M),$$

quantifies the effective dimensionality of the module in terms of the dimensions of the *image* of the linear function encoded in M . Thus, a low rank indicates that vectors are nearly co-linear (a one-dimensional flow), whereas a high rank implies multi-directional or isotropic behavior. In biological terms, R measures the number of independent directions of functional or regulatory diversification within the module.

17.2.3 Principal Flow Direction

The dominant orientation of a module can be extracted in two closely related, but conceptually distinct ways. Let

$$M \in \mathbb{R}^{d \times n}$$

denote the matrix of module vectors, and

$$C = MM^\top$$

the corresponding covariance-like operator formed from the outer products of these vectors.

17.2.3.1 Geometric Principal Axis (from SVD of M). The left singular vectors of M obtained from

$$M = U\Sigma V^\top$$

capture the directions along which the *vectors themselves* exhibit maximal geometric spread in ambient space. The first singular vector,

$$\vec{u}_1 = \text{svdvec}_1(M),$$

describes the orientation of greatest variance among the coordinates of the module vectors. This direction answers questions of the form:

“Along which axis do the module’s vectors *lie* or *spread out* the most?”

For gene expression modules, this corresponds for example to the dominant tissue or condition axis that best separates the expression patterns contained in the module.

17.2.3.2 Dynamical Principal Flow (from EigenDecomposition of C). Since

$$C = MM^\top = U\Sigma^2U^\top,$$

the eigenvectors of C coincide exactly with the left singular vectors of M , while their eigenvalues are the squared singular values. Thus,

$$\vec{f} = \text{eigvec}_1(C)$$

selects the same direction as $\vec{u}_1$, but interprets it through the lens of the *force-like* operator $C = \sum_j \vec{v}_j \vec{v}_j^\top$. In this formulation, vectors with large magnitude contribute disproportionately more, causing $\vec{f}$ to represent the direction along which a virtual particle would experience the strongest net push from the entire module. This dynamical perspective answers questions such as:

“In which direction does the module collectively *pull*, *push*, or *bias* the system?”

In transcriptomic modules, this flow direction reflects the dominant tissue or condition toward which the expression vectors are collectively drifting; in socioeconomic trajectories, it identifies the main axis along which countries in the module exhibit consistent directional change.

17.2.3.3 Interpretative Complementarity. Although the eigenvectors obtained from M and C coincide, they express different scientific concepts: the SVD of M reveals the *geometric orientation* of the module (where the vectors reside), while the eigendecomposition of C reveals the *dynamical orientation* (what the vectors collectively do). These two interpretations can diverge substantially in practice. A module may show low geometric spread but exert a strong directional push, or conversely, may occupy a broad geometric region while exerting little coherent dynamical flow. Distinguishing these cases is essential for understanding both structural and functional aspects of high-dimensional vector fields in Tensor Omics.

17.2.3.4 Mathematical Note on Coincidence of Eigenvectors (singular vectors) Since the singular value decomposition of the module matrix satisfies

$$M = U\Sigma V^\top,$$

the operator $C = MM^\top$ takes the form

$$C = U\Sigma^2 U^\top.$$

Thus the eigenvectors of C are exactly the left singular vectors of M , differing at most by a sign. In particular, the first eigenvector of C and the first left singular vector of M point in the same direction in ambient space, yielding a cosine similarity of ± 1 . Throughout Tensor Omics we refer to such vectors as *coinciding*, since their orientation is immaterial and only their direction in the vector space is conceptually relevant.

17.2.4 Rotation Axis and Magnitude

The rotation contained in a module cannot be extracted from $C = MM^\top$, since C is symmetric and therefore encodes no antisymmetric component. Instead, rotational behaviour is obtained from the orthogonal factor of the polar decomposition of the empirical linear operator M . Using the singular value decomposition,

$$M = U\Sigma V^\top,$$

the polar factors are given by

$$R = UV^\top, \quad S = V\Sigma V^\top,$$

where R is orthogonal and represents the pure rotational (and possibly reflective) component of the module, while S contains all symmetric stretching.

To obtain a genuine rotational generator we take the matrix logarithm of R ,

$$A = \log(R),$$

which is guaranteed to be skew-symmetric for any orthogonal matrix with determinant $+1$. The eigenvalues of A occur in purely imaginary conjugate pairs $\pm i\omega$, and the dominant rotational frequency is defined as

$$\omega_{\max} = \max_i |\operatorname{Im}(\lambda_i(A))|.$$

This quantity measures the angular velocity associated with the principal rotational mode of the module.

The eigenvectors corresponding to the pair $\pm i\omega_{\max}$ span the *principal rotation plane*. In three dimensions this plane collapses to a classical rotation axis; in higher dimensions it identifies a distinguished two-dimensional subspace in which the dominant twisting or swirling motion occurs. When visualised in reduced projection spaces (e.g. the RAP), this plane is represented by a canonical axis or arrow indicating the orientation of rotation.

Together, the rotation magnitude $\omega_{\max}$ and the principal rotation plane provide a second, fully independent descriptor of the module's internal organisation. They distinguish coherent cyclic or twisting behaviour in the vector field from purely divergent, convergent, or noise-dominated structure, thereby complementing the principal flow direction extracted from the symmetric component.

17.2.4.1 Approximate Rotation via the Antisymmetric Component of M

In settings where computational resources are limited, an approximate measure of rotational tendency can be obtained directly from the antisymmetric part of the module matrix,

$$A_{\text{approx}} = \frac{1}{2}(M - M^\top).$$

This operator is always skew-symmetric and therefore admits purely imaginary eigenvalues, which provide a fast surrogate for rotational strength:

$$\omega_{\text{approx}} = \max_i |\operatorname{Im}(\lambda_i(A_{\text{approx}}))|.$$

The corresponding eigenvector pair spans an approximate rotation plane.

Unlike the polar-decomposition method, this approximation does not isolate a genuine rotational component of the underlying linear operator and may conflate rotation with shear or asymmetric stretching. Nevertheless, it is inexpensive to compute, requires no matrix logarithm, and often provides a qualitatively useful indicator of swirling or twisting tendencies when analysing large numbers of modules or performing rapid exploratory screening.

17.3 Conclusion of the four module descriptors

Together, these four quantities — energy, rank, principal direction, and rotation — form the Tensor Omics Module Descriptors (Mod-4-Desc).

17.4 Normalization of module descriptors

To enable comparison across subfields of varying size k , the signal strength can be normalized by the root of the cardinality:

$$\text{RFI} = \frac{\|M\|_F}{\sqrt{k}}$$

This Relative Frobenius Index (RFI) provides a density-independent measure of average vector magnitude per subfield.

Note: All direction vectors $\vec{v} \in \mathbb{R}^n \setminus \{\vec{0}\}$ are assumed to be non-zero, as normalization and angular analysis are undefined for the zero vector. If an unscaled antisymmetric component $A = C - C^T$ is used instead of the factorized form, this corresponds to measuring the total (unweighted) rotation strength. The scaled form $\frac{1}{2}(C - C^T)$ is preferred for decomposition consistency and interpretability.

17.5 Vector Sets versus Vector Fields

A *vector set* (e.g. all expression vectors of a gene family) consists only of vectors and their distribution in space.

When vectors additionally carry explicit coordinates—as in shift vectors originating at gene-family centroids—they form a *vector field*. In such cases, Mod-4-Desc can be applied either to the coordinates or to the vectors themselves, yielding complementary information:

- **On the coordinates $\vec{x}_i$:** the descriptors characterize the geometry and spatial extent of the region in Tensor Omics space, answering questions like:
 - Which semantic axes dominate the spatial extent of the field?
 - Is the field elongated or isotropic (rank)?
 - Does the region itself curve or spiral through Tensor space (rotation)?
- **On the vectors $\vec{v}_i$:** the descriptors characterize the dynamics or changes that occur within that region:
 - Which semantic axes dominate expression changes?
 - How strong are those changes (energy)?
 - Are changes co-directed or diverse (rank)?
 - Do they form coherent rotational flows or transitions (rotation)?

Examples: In transcriptomics, coordinates correspond to gene-family positions in expression space, while shift vectors represent changes such as stress responses. In physics, coordinates represent spatial positions, and vectors represent wind velocities or magnetic field directions.

17.6 Tracing a Module along its Semantic Axis

Assume a module has an associated *semantic axis* $\vec{d}_{\text{sem}}$ (see Section 12.8 for its definition), obtained for example by the Mjöltnir manifold learning method. We can then analyze how Mod-4-Desc quantities evolve along this axis.

A Gaussian kernel $K(\vec{x}; \sigma)$ is moved along $\vec{d}_{\text{sem}}$. At each kernel center, the local subset of vectors $\{\vec{v}_i\}$ within the kernel’s support is collected into a matrix M , and the four TOX Field Descriptors are computed locally:

$$\{E(\vec{x}), R(\vec{x}), \vec{f}(\vec{x}), \omega(\vec{x})\}.$$

Plotting these quantities along the semantic axis reveals how field energy, diversity, and rotational behavior change over semantic space (e.g., developmental time, aging trajectory, or stress gradient). This procedure generalizes naturally to 2D semantic surfaces, where the descriptors can be visualized as tensor fields over the manifold.

17.7 Interpretation and Scientific Questions

These descriptors allow a model-free geometric interpretation of high-dimensional data:

- **Energy:** How strong are the local signals or shifts?
- **Rank:** How many independent biological factors are active?
- **Flow:** In which semantic direction does the process move?
- **Rotation:** Are changes linear, oscillatory, or cyclic?

Together, they enable systematic comparison of complex omics or physical fields, revealing patterns of coherence, divergence, and functional rotation in spaces where traditional differential or statistical models fail.

18 Field Surface and Volume Inference (FiSure, “Eitri”)

18.1 Motivation and Concept

To compare subfields or regions within Tensor Omics space, we often need to quantify their *geometric extent* — the shape, surface, and volume occupied by a field of vectors. The **Field Surface and Volume Inference (FiSure)** method provides a model-free procedure to estimate these quantities directly from the coordinates and projections of field vectors.

Conceptually, FiSure treats a field as a set of vectors $\{\vec{x}_i\}$ distributed around a *semantic axis* or *handle* (e.g. a Mjöltnir manifold), and reconstructs the outer surface that encloses them. The method is analogous to measuring the radial extent of a magnetic flux tube or a fluid jet along its main trajectory.

18.2 Algorithmic Outline

Assume a learned one-dimensional semantic axis $\vec{h}(t) \in \mathbb{R}^n$ (the **handle**) representing the main trajectory of the field. For a field consisting of coordinates $\{\vec{x}_i\}_{i=1}^m$, FiSure proceeds as follows:

1. **Projection onto the Handle.** Each field point is projected onto the handle by nearest-point projection:

$$t_i = \arg \min_t \|\vec{x}_i - \vec{h}(t)\|.$$

This yields the axial coordinate t_i of each field vector.

2. **Handle Segmentation.** The handle is divided into k equal-percentile segments $\{[t_j, t_{j+1}]\}$, such that each segment corresponds to an equal fraction of projected vectors. Each segment is approximated by a straight line between its endpoints $\vec{h}(t_j)$ and $\vec{h}(t_{j+1})$.

3. **Local Coordinate Frame.** For each segment, a local cylindrical coordinate system is constructed:

$$\text{axis vector: } \vec{a}_j = \frac{\vec{h}(t_{j+1}) - \vec{h}(t_j)}{\|\vec{h}(t_{j+1}) - \vec{h}(t_j)\|}.$$

Orthogonal basis vectors are defined as $\{\vec{u}_j, \vec{v}_j\}$, spanning the plane perpendicular to $\vec{a}_j$.

4. **Angular Sampling.** The circular cross-section around the segment is divided into c angular bins:

$$\theta_k = \frac{2\pi k}{c}, \quad k = 1, \dots, c.$$

A fixed global reference axis (e.g. the first axis of Tensor Omics space) is used to align the angular bins between segments.

5. **Radial Extent and Surface Extraction.** For each angular bin, FiSure computes the radial distance of all vectors whose projections fall within the segment and angle range:

$$r_{j,k} = \max_{\vec{x}_i \in \mathcal{S}_{j,k}} \|(\vec{x}_i - \vec{h}(t_j)) - \langle \vec{x}_i - \vec{h}(t_j), \vec{a}_j \rangle \vec{a}_j\|.$$

The point attaining this maximum is recorded as the local surface point:

$$\vec{p}_{j,k} = \vec{h}(t_j) + r_{j,k} (\cos \theta_k \vec{u}_j + \sin \theta_k \vec{v}_j).$$

The corresponding outward normal vector is

$$\vec{n}_{j,k} = \frac{\vec{p}_{j,k} - \vec{h}(t_j)}{\|\vec{p}_{j,k} - \vec{h}(t_j)\|}.$$

6. **Surface and Volume Assembly.** The set of surface points $\mathcal{P} = \{\vec{p}_{j,k}\}$ defines a discrete surface mesh. The local volume of the segment between t_j and t_{j+1} can be approximated as a truncated cylinder:

$$V_j \approx \frac{\pi \Delta s_j}{c} \sum_{k=1}^c \frac{r_{j,k}^2 + r_{j+1,k}^2}{2},$$

where $\Delta s_j = \|\vec{h}(t_{j+1}) - \vec{h}(t_j)\|$. Summing over all segments yields the total field volume:

$$V_{\text{field}} = \sum_{j=1}^{k-1} V_j.$$

18.3 Interpretation of FiSure Quantities

FiSure yields several interpretable geometric measures:

- **Surface Vectors:** Each $(\vec{p}_{j,k}, \vec{n}_{j,k})$ pair represents a point on the field's boundary and its outward orientation. These can be analyzed as a secondary *surface vector field*, quantifying how the field opens, closes, or bends in Tensor space.
- **Local Volume Elements:** The segment volumes V_j allow normalization of other descriptors, such as energy or rank, by the spatial size of the field.
- **Field Morphology:** Variations in $r_{j,k}$ and curvature of the handle describe anisotropy and torsion of the field. A stable $r_{j,k}$ profile implies a cylindrical, consistent field; strong local expansion or contraction indicates field divergence or collapse.

18.4 Applications and Interpretation

FiSure provides a direct geometric connection between field structure and function. In the context of Tensor Omics, examples include:

- Measuring the **semantic volume** occupied by gene families associated with aging or stress responses.
- Quantifying whether such semantic volumes **expand, contract, or twist** along a developmental or disease axis.
- Identifying rotational or asymmetrical **surface deformations** that indicate functional bifurcation (e.g., tissue-specific reprogramming).

The same principles apply to physical or environmental fields: for a wind field, FiSure reconstructs the shape of a coherent airflow tube; for a magnetic field, it delineates regions of strong magnetic flux concentration.

18.5 Summary

FiSure extends Tensor Omics by providing a quantitative method for reconstructing the **shape** and **extent** of a field. By defining surfaces and volumes empirically from vector distributions, it bridges local field descriptors (energy, rank, rotation) with global geometric form, completing the description of high-dimensional semantic dynamics in Tensor space.